



OFFCHAIN SECURITY ASSESSMENT

Pear

— **VAULT SERVICE**

SECURITY REVIEW

10 SEPTEMBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1	About Shieldify	3
2	Disclaimer	3
3	About Pear	3
4	Risk Classification	4
4.1	Impact	4
4.2	Likelihood	4
5	Security Review Summary	4
5.1	Protocol Summary	5
5.2	Scope	5
6	Findings Summary	6
7	Findings	7

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Pear

Pear is a system of ERC-4626 vaults on HyperEVM where investors pool capital and a designated leader runs delegated pair-trading on Hyperliquid, without ever taking custody of investor funds.

Capital lives on-chain in two places: the vault contract itself on the EVM side, and a Privy-custodied agent wallet that holds the trading position. Trade execution happens off-chain - `pear-vault-service` handles signing, and the `pear-pro` / `pear-hl-engine` trading APIs route the orders. The leader can therefore direct strategy without ever being able to move investor principal out of the system.

At a high level, the main functionalities are:

- Investors deposit an asset token into a vault and receive ERC-4626 shares
- A leader directs delegated pair-trading on Hyperliquid through an agent wallet bound to the vault
- Share price is derived from a composite net-asset-value that spans the vault's EVM balance, the agent's wallet balance, the agent's HyperCore perp and spot positions, a Felix vault position, and assets in flight across the EVM-HyperCore bridge
- Exits run through two paths: an immediate direct withdrawal when the vault holds enough spare liquidity, or a withdrawal queue settled later by an admin

- A Comptroller holds protocol-wide configuration – vault configs, fee parameters, and per-user deposit caps derived from PEAR tiers
- A factory deploys and registers vaults, each bound to its own agent

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	 CRITICAL	 HIGH	 MEDIUM
Likelihood: Medium	 HIGH	 MEDIUM	 LOW
Likelihood: Low	 MEDIUM	 LOW	 LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

This review covers the off-chain vault service, complementing the earlier engagements on the Pear vault smart contracts.

The security review lasted 7 days with a total of 112 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-structured and the service's own architecture and API documentation set out clear trust boundaries. The audit report contributed by identifying two Critical, one High, six Medium and eight Low severity issues. They're mainly related to **authorization and access control, custody and fund-accounting logic, and the reliability of the off-chain infrastructure and indexing layer.**

The Pear team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	PEAR VAULT OFF-CHAIN SERVICE
Repository	pear-vault-service
Type of Project	Off-Chain Vault Custody, Signing & Reporting Service
Security Review Timeline	7 days
Review Commit Hash	2146d619466025a16a38f0ea387d14229b324614
Fixes Review Commit Hash	8117ea32bc51fb73773d3a4dfe96854a7c2ce3e8

5.2 Scope

FOLDER	RESPONSIBILITY
<code>src/vaults/</code>	Vault creation, agent rotation, trader assignment, lookup
<code>src/signing/</code>	EIP-712 signing via the Privy master key; Privy SDK wrapper
<code>src/agent/</code>	ERC-20 approvals and swap-and-transfer on behalf of the vault
<code>src/vault-api-wallet/</code>	Hyperliquid agent (API wallet) approval
<code>src/bridge/</code>	HyperCore ↔ HyperEVM transfers and withdrawals
<code>src/hyperliquid/</code>	Hyperliquid exchange and info client wrapper
<code>src/events/</code>	Deposit/withdrawal log indexer and history endpoints
<code>src/nav/</code>	NAV poller, multicall reader, NAV endpoints
<code>src/chain/</code>	HyperEVM chain definition and viem clients
<code>src/auth/</code>	Consumer JWT guard and internal API-key guard
<code>src/config/</code>	Environment and consumer-registry schemas
<code>src/health/</code>	Liveness and dependency reporting
<code>src/main.ts</code>	Global pipes, middleware, module wiring
<code>prisma/</code>	Schema and migrations

FOLDER	RESPONSIBILITY
app.module.ts	
main.ts	

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Critical** and **High** issues: **3**
- **Medium** issues: **6**
- **Low** issues: **8**
- **Info** issues: **2**

ID	TITLE	SEVERITY	STATUS
[C-01]	Cross-Vault Privy Signer Substitution: Public + Attacker-Writable <code>privyId</code> Enables Agent-Wallet Takeover	CRITICAL	FIXED
[C-02]	Unconstrained Signing Surface Lets a Trader Sign Arbitrary <code>UsdSend</code> , Draining ~90% of Vault Capital	CRITICAL	FIXED
[H-01]	Persistent Trader-Created Hyperliquid Agent Authority Survives Trader Removal and Vault Deactivation	HIGH	FIXED
[M-01]	Non-Atomic Custody Workflows: Partial Failure Strands Funds and Orphans Vault State	MEDIUM	FIXED
[M-02]	<code>swap-and-transfer</code> Moves the Total USDC Balance Rather Than Sale Proceeds	MEDIUM	FIXED
[M-03]	Events Indexer Has No Reorg Protection or Confirmation Depth	MEDIUM	FIXED
[M-04]	NAV Subsystem Is Entirely Non-Functional: <code>multicall3</code> Is Not Configured on the HyperEVM Chain Object	MEDIUM	FIXED
[M-05]	Events Indexer Has No Per-Vault Backfill: Migrated Vaults Miss Their Pre-Insertion History	MEDIUM	FIXED
[M-06]	Raw Prisma <code>BigInt</code> Values Make Non-Empty Transaction-History and <code>nav/latest</code> Responses Unserializable	MEDIUM	FIXED
[L-01]	Internal API Key Compared in Non-Constant Time (Timing Side-Channel)	LOW	FIXED

ID	TITLE	SEVERITY	STATUS
[L-02]	Unvalidated <code>.vaultAddress</code> Path Param Reaches <code>getAddress()</code> and Returns HTTP 500	LOW	FIXED
[L-03]	Unauthenticated Reads of Transaction History and NAV; NAV Range Unbounded	LOW	ACKNOWLEDGED
[L-04]	<code>initialUsdcDeposit</code> Is Accepted and Validated on Vault Creation, Then Silently Ignored	LOW	FIXED
[L-05]	Poll Intervals Above 59 Seconds Are Silently Ignored and Misreported in Logs	LOW	FIXED
[L-06]	Vault Service Applies No Independent JWT Age Cap or Revocation	LOW	FIXED
[L-07]	No Code-Side Rate Limiting on Operator-Funded Vault Creation	LOW	FIXED
[L-08]	Upstream Provider Error Text Is Returned Verbatim in API Responses	LOW	FIXED
[I-01]	<code>SignTransferDto.amount</code> Bypasses Decimal Validation, Silently Signing Zero-Value Transfers	INFO	FIXED
[I-02]	<code>/health</code> Reports <code>privy_connected</code> Without Ever Contacting Privy	INFO	FIXED

7 Findings

C-01

CRITICAL FIXED

Cross-Vault Privy Signer Substitution: Public and Attacker-Writable `privyId` Enables Agent-Wallet Takeover

Description

Authority over a managed (Privy) signing wallet is keyed by the `privyId` column on the `vaults` row, but that column is **(a) publicly disclosed** and **(b) attacker-writable on a row the attacker legitimately owns**. Nothing binds a `privyId` to the vault that provisioned it, so an attacker can copy a victim's `privyId` onto their own vault and then exercise every owner-gated wallet operation against the victim's wallet. The owner-or-trader authorization check is never bypassed – it is evaluated against a row the attacker is permitted to rewrite.

Three independent defects compose:

1. **Disclosure.** `GET /vaults/vaultAddress` and `GET /vaults` carry no `@UseGuards`, no `select`, and there is no `ClassSerializerInterceptor` registered (`app.module.ts`, `main.ts`). The raw Prisma row – including `privyId` – is returned to an anonymous caller.
2. **Attacker-writable authority key.** `PATCH /vaults/vaultAddress/agent` (`rotateAgent`) accepts `privyId` as a bare `@IsString()` and writes it verbatim. `loadAsOwner` only checks that the caller owns *that* vault. There is no ownership/uniqueness/"already bound elsewhere" check, and no `@unique` on `privy_id` in the schema to stop it at the DB.
3. **Authority derived from the mutable field.** Every fund-touching service (`signing.service.ts`, `agent.service.ts`, `vault-api-wallet.service.ts`) loads the row by `vaultAddress`, checks the caller against `ownerAddress`, then acts on that row's `privyId`. Once step 2 has swapped in the victim's `privyId`, these all operate on the victim's wallet.

The disclosure breaks an invariant the service's own architecture document states

`docs/ARCHITECTURE.md#207` records the intended handling of this exact value:

Privy keys are the highest-value secret. Never logged, never returned in any response.

Both halves of that sentence are contradicted by the implementation:

- **Returned in responses.** `findByAddress` (`vaults.service.ts#148-154`) and `list` (`:184-196`) return the raw Prisma row through unguarded `GET` routes, and `create` (`:141`) returns `privy_id` as an explicit, named response field – so the disclosure is not merely an oversight in serialization, it is written out deliberately in one path.
- **Logged.** `privy.client.ts#51` logs `Privy wallet provisioned: id=${wallet.id} address=${wallet.address}` at `log` level on every vault creation, and `:75` / `:123` log the id again on every signing and transaction failure. The wallet id is therefore written to application logs during normal operation, not only on error.

This matters beyond documentation drift. The HTTP disclosure and the log disclosure are **distinct channels with different exposure populations**: the log channel hands `privyId` to anyone with access to log output – the log aggregation platform and its vendor, SRE and support staff, anything shipping or backing up logs, and any downstream system that ingests them – none of whom need to reach the service's HTTP surface at all. Remediating only the HTTP responses (Recommendation 2) leaves this channel fully open.

Location of Affected Code

File: [src/vaults/vaults.controller.ts#L36-L39](#)

File: [src/vaults/vaults.controller.ts#L41-L44](#)

File: [src/vaults/vaults.service.ts#L141](#)

File: [src/vaults/vaults.service.ts#L148-L154](#)

File: [src/vaults/vaults.service.ts#L222-L241](#)

File: [src/vaults/dto/rotate-agent.dto.ts#L8-L10](#)

File: [src/signing/signing.service.ts#L34-L66](#)

File: [src/signing/privy.client.ts#L51](#)

File: [src/signing/privy.client.ts#L75](#)

File: [src/signing/privy.client.ts#L123](#)

File: [prisma/schema.prisma#L26](#)

File: [docs/ARCHITECTURE.md#L207](#)

Impact

Full takeover of any victim vault's **agent wallet** from an ordinary user account, with no privileged secret. An attacker can sign arbitrary Hyperliquid actions as the victim (including `UsdSend` withdrawals), install their own EOA as a persistent Hyperliquid agent, and grant arbitrary ERC-20 approvals on HyperEVM. Loss is bounded to what the agent wallet controls – the Hyperliquid Core (perp/spot) and HyperEVM balances it operates, which during active trading is the vault's working capital. This does **not** by itself move principal held in the PearVault contract; it compromises the agent-wallet custody boundary. The `/sync/migrate-vault` overwrite is the same root cause reached through a path that additionally requires `INTERNAL_API_KEY`.

The log-disclosure channel widens the set of parties who can obtain a `privyId` beyond callers of the HTTP API to anyone with access to application logs or the systems that store, ship, or index them – a population that is typically broader, longer-lived, and less access-controlled than the API surface, and one for which the value's sensitivity is not obvious from the log line itself.

Note also that the service's own documentation designates the agent wallet as the destination for vault funds (`docs/api.md:32` – `transfer-to-hypercore` moves *"Vault (HyperEVM) → agent HyperCore"*, `docs/ARCHITECTURE.md:131` – the `pending_activation` state exists to *"give the leader a chance to fund the agent before it's authorized to trade"*). The agent wallet holding live capital is therefore the designed steady state, not an edge case. The client has confirmed the proportion: **~90% of a vault's capital sits in the HyperCore agent account** (10% in the PearVault contract), so the reachable loss on takeover is up to ~90% of vault capital.

Proof of Concept

1. Attacker is a normal Pear user (holds a valid consumer JWT for their own address) and creates one vault of their own through the standard flow. They do **not** hold `INTERNAL_API_KEY`, `PRIVY_APP_SECRET`, the victim's JWT, or any DB access.
2. Attacker calls `GET /vaults/{victimVault}` with no auth and reads the victim's `privyId`. *(Alternative path: any party with read access to application logs obtains the same value from the provisioning or error lines above, without touching the API.)*
3. Attacker calls `PATCH /vaults/{attackerVault}/agent` with `{"privyId": "<victim privy_id>"}`. `loadAsOwner` passes (attacker owns that vault); the attacker's row now points at the victim's wallet. The victim's row is untouched – nothing alarms.
4. Attacker calls owner-gated routes on their **own** vault, which now execute against the victim's wallet:
 - `POST /vaults/{attackerVault}/sign` – arbitrary EIP-712 signed by the victim wallet (e.g. `HyperliquidTransaction:UsdSend` to an attacker address);
 - `POST /vaults/{attackerVault}/api-wallet/approve` – victim wallet approves the **attacker's own EOA as a Hyperliquid agent** (persistent control);
 - `POST /vaults/{attackerVault}/agent/approve` – victim wallet grants an ERC-20 approval to an attacker-chosen spender.

Recommendation

1. **Stop accepting a caller-supplied** `privyId`. Remove `privyId` from `RotateAgentDto` and `rotateAgent`; an agent rotation should provision a fresh Privy wallet server-side and never bind an id supplied by the caller.
2. **Stop returning** `privyId`. Replace the raw Prisma row in `findByAddress`, `list`, `checkAuthz`, and the `create` response with an explicit `select` allowlist.

Team Response

Fixed.

C-02

● CRITICAL

FIXED

Unconstrained Signing Surface Lets a Trader Sign Arbitrary `UsdSend`, Draining ~90% of Vault Capital

Description

The signing endpoints (`POST /vaults/vaultAddress/sign`, `/sign/order`, `/sign/transfer`) apply **no constraint on the typed data they sign**. The DTO accepts any `primaryType`, any `domain`, and any `message` (`sign.dto.ts:16-26`), and the service forwards the payload to the vault's Privy master key verbatim (`signing.service.ts:68-105`). There is no allowlist of signable `primaryType`s, no domain binding, and no destination/amount policy.

Authorization on these routes admits the vault **owner or trader** (`signing.service.ts:43-45`). Every actual fund-movement endpoint, by contrast, is **owner-only**:

- `bridge.withdraw` / `withdrawToEvm` / `spotToPerp` / `perpToSpot` → `loadAsOwner` (`bridge.service.ts:36`)
- `agent.approve` / `swap-and-transfer` → `loadVaultAsOwner` (`agent.access.ts:7`)

So the trader is, by the application's own gating, a delegate who is **not** permitted to move funds. The unconstrained signing surface lets the trader escape that restriction: the trader can hand `/sign` a `HyperliquidTransaction:UsdSend` and obtain the vault master's signature over a fund transfer to an arbitrary destination, then submit it to Hyperliquid directly - no further access to the vault service required. The vault's Privy wallet is the Hyperliquid account that holds the funds (it is the address queried for balances in `hyperliquid.service` `getSpotBalances` / `getPerpBalances(agentAddress)`), so a `UsdSend` signed by that key is a real transfer of the account's USDC.

Trust boundary (confirmed by the client). This finding does not assume a malicious owner/admin. The trader is a distinct, *lower-privileged* delegate - the client has confirmed the trader wallet "is meant to be strictly lower-privileged than the vault owner" and is a changeable role (the leader wallet cannot be changed; the trader wallet can). The intended boundary is documented in the API itself: `api.md` marks `/sign/transfer` as **Leader-only** (line 78) while `/sign` and `/sign/order` are **Trader-or-engine** (lines 76-77) - i.e. traders are meant to sign *orders*, not *transfers*. The defect: because the trader-accessible `/sign` route applies **no** `primaryType` **allowlist**, a trader can sign the exact fund-moving payload (`UsdSend`, an ERC-20 transfer envelope) that `/sign/transfer` is Leader-gated against - trivially defeating the documented boundary. (Separately, the code lets the trader hit `/sign/transfer` directly too, since `resolveAndAuthorize` is owner-or-trader, contradicting `api.md`'s Leader-only marking.) An owner performing the same signing is out of scope; this is reported solely as trader→withdrawal privilege escalation.

Three folded facets of the same root cause (unconstrained signing surface):

- **No primaryType allowlist** – the same route that signs legitimate Hyperliquid orders will sign a `UsdSend` withdrawal.
- **Optional payload on /sign/transfer** (`sign.dto.ts:58-61`) – an arbitrary envelope bypasses the "structured transfer" shape.
- **No domain binding** – the default envelope omits `chainId` / `verifyingContract` (`signing.service.ts:125`), and caller-supplied domains are never validated, so signatures are not bound to a chain or contract.

Location of Affected Code

File: [src/signing/signing.service.ts#L43-L45](#)

File: [src/signing/signing.service.ts#L68-L105](#)

File: [src/signing/signing.service.ts#L125](#)

File: [src/signing/dto/sign.dto.ts#L16-L26](#)

File: [src/signing/dto/sign.dto.ts#L58-L61](#)

File: [src/bridge/bridge.service.ts#L36](#)

File: [src/agent/agent.access.ts#L7](#)

Impact

Privilege escalation from a delegated trader to full fund extraction. **Precondition:** the attacker holds the vault's owner-appointed trader role (set via `POST /vaults/vaultAddress/trader`, owner-only) – i.e. a compromised or rogue trader, not an arbitrary anonymous caller. Given that role, the trader can obtain the vault master's signature over arbitrary Hyperliquid actions, including a `UsdSend` that drains the account's USDC to an attacker-controlled destination. This defeats the owner/trader separation the codebase enforces everywhere else and compromises the vault's HyperCore (perp/spot) balance, which during active trading is the vault's working capital. The missing domain binding additionally leaves any produced signature unbound to a chain or contract, widening cross-context replay exposure.

Impact magnitude (confirmed): the client states **~90% of a vault's capital sits in the HyperCore agent account** (10% in the PearVault contract). Since the agent account is exactly what a `UsdSend` signed by the Privy master drains, the reachable loss is up to ~90% of vault capital (the withdrawable portion directly; capital in open positions after unwinding them via the same signing route).

Realized impact is theft of the majority of vault funds. Two preconditions apply:

- *Requires the owner-appointed trader role* – a lower-privileged, client-confirmed-changeable delegate; a rogue or compromised trader is a realistic attacker, and the intended boundary (`api.md` `/sign/transfer` = Leader-only) is one the code already violates.
- *On-exchange submission of the signed payload follows from the signature itself and was traced through the code rather than executed against Hyperliquid.*

This is distinguished from C-01 (Cross-Vault Privy Signer Substitution) only by attacker set: that finding requires no special role, no `privyId` rebinding, and no leaked secret, making it the broader-reach of the two, while this one is reachable by the trader. The two escalations are independent.

Proof of Concept

1. A vault has an owner (leader) and a trader set by the owner via `POST /vaults/vaultAddress/trader`. The trader holds a valid consumer JWT for their own address – the normal delegated-trading credential.
2. The trader calls `POST /vaults/vaultAddress/withdraw` → **403** (`loadAsOwner`; the trader is not the owner). The application intends the trader to be unable to withdraw.
3. The trader instead calls `POST /vaults/vaultAddress/sign` with a `HyperliquidTransaction:UsdSend` payload whose `destination` is the trader's own address and `amount` is the account balance. `resolveAndAuthorize` passes (caller is the trader), and the vault master signs it.
4. The trader submits the signed `UsdSend` to Hyperliquid, transferring the vault account's USDC to their own address. The owner-only withdrawal gate has been bypassed via the signing route.

Recommendation

1. **Allowlist what may be signed.** Reject any `primaryType` outside an explicit set of intended, non-fund-moving actions, and explicitly deny `UsdSend`, `Withdraw`, and `spotSend` on the signing routes.
2. **Gate fund-capable signing to the owner.** Restrict `/sign/transfer`, and any route able to express a fund movement, to the vault owner – matching the boundary already enforced in `BridgeService` and `AgentService`.

Team Response

Fixed.

Persistent Trader-Created Hyperliquid Agent Authority Survives Trader Removal and Vault Deactivation

Description

A vault **trader** – a delegate the application otherwise restricts below the owner – can cause the vault's Privy master to grant Hyperliquid **agent authority** to an API wallet address of the trader's choosing (`approveApiWallet` is owner-or-trader, `vault-api-wallet.service.ts:41-48,84-92`). The granted authority lives **on Hyperliquid**, attached to a credential (the approved EOA) that acts directly against the exchange and needs no further access to the vault service.

The owner's remediation levers do not withdraw that authority:

- There is **no revoke/de-authorize primitive** anywhere on the Hyperliquid path – the wrapper exposes `approveAgent` / `approveBuilderFee` only (`hyperliquid.service.ts:130-146`). The service *cannot* revoke even if it wanted to.
- `setTrader` (removing the trader) and `deactivateAgent` (deactivating the vault) are **DB-only row updates** that never touch Hyperliquid (`vaults.service.ts:198-220`).
- `is_approved` is set true on first approval and **never reset** (`vault-api-wallet.service.ts:69-71`).
- Deactivating the vault does not help and slightly hurts: the only implicit displacement path – re-approving a fresh address under the same Hyperliquid `agentName` – is blocked once the vault is inactive (`VAULT_INACTIVE`), so deactivation removes the owner's last lever to rotate the agent away.

The one implicit mitigation is caller-controlled, so grants accumulate

The service's own comment at `vault-api-wallet.service.ts:26-29` describes the sole containment property this design relies on:

HL revokes any previous agent with the same `agentName`, so calling this with a new address is also the rotation path.

That property holds only while the name is held constant – and **nothing holds it constant**.

`agentName` is resolved as `input.agentName ?? input.serviceName ?? DEFAULT_AGENT_NAME` (`:42`), where both `agentName` and `serviceName` come straight from the request body and are validated only as non-empty strings (`dto/approve.dto.ts:20-30`). There is no allowlist, no binding to the authenticated consumer, and no check against names already used on this vault.

A caller who simply varies the name on each call therefore does not rotate anything – each approval installs an **additional, concurrent** Hyperliquid agent, all of them signed by the vault's Privy master, all of them retaining authority. The single fact that makes each individual grant un-revokable (no revoke primitive) compounds: the trader is not limited to planting one un-revokable agent, but as many as they care to request, and each survives trader removal and vault deactivation independently.

This also degrades the audit trail intended to compensate. The DTO comment at [dto/approve.dto.ts:16-19](#) states `serviceName` is *"logged for audit"*, and [:44-46](#) writes it to the application log – but the value is attacker-supplied, so a trader can label their own grants as `pear-hl-engine` (or anything else) and the log records the attacker's chosen string as though it identified the calling service.

Trust boundary (confirmed by the client). This does not assume a malicious owner. The client has confirmed the trader is meant to be strictly lower-privileged than the owner, and that the **trader wallet is changed as a matter of normal operation** ("we can change trader wallet but not the leader wallet"). That trader-rotation is exactly the remediation lever this finding shows to be ineffective: changing the trader is a DB-only update that leaves the departing trader's Hyperliquid agent grants live on-exchange. The account those grants control holds **~90% of the vault's capital** (client-confirmed), so the persistent trading authority is over the majority of vault funds.

Location of Affected Code

File: [src/vault-api-wallet/vault-api-wallet.service.ts#L41-L48](#)

File: [src/vault-api-wallet/vault-api-wallet.service.ts#L42](#)

File: [src/vault-api-wallet/vault-api-wallet.service.ts#L69-L71](#)

File: [src/vault-api-wallet/vault-api-wallet.service.ts#L84-L92](#)

File: [src/vault-api-wallet/dto/approve.dto.ts#L20-L30](#)

File: [src/hyperliquid/hyperliquid.service.ts#L130-L146](#)

File: [src/vaults/vaults.service.ts#L198-L204](#)

File: [src/vaults/vaults.service.ts#L214-L220](#)

Impact

A trader can establish persistent, out-of-band control of the vault's Hyperliquid account that survives the owner's removal of the trader and deactivation of the vault. A Hyperliquid agent can place orders (not withdraw), so the loss vector is continued trading control – value bleed via adverse fills or self-trading against an attacker-controlled counterparty – rather than direct withdrawal. The exposure is bounded by Hyperliquid's native agent-authority lifetime, but within that window the owner's in-product remediation is ineffective. **Precondition:** the attacker holds the owner-appointed trader role.

Because the `agentName` displacement path is caller-controlled, the exposure is not a single stale grant but an arbitrary number of concurrent ones, established in a single session before the owner has any opportunity to react. Revocation, once implemented, must therefore enumerate and withdraw **all** outstanding grants for a vault rather than the most recent one, and the service currently stores no record of which names or addresses it has approved – so the set is not recoverable from the service's own state.

Proof of Concept

1. The owner appoints a trader (`POST /vaults/vaultAddress/trader`).
2. The trader calls `POST /vaults/vaultAddress/api-wallet/approve` with `apiWalletAddress` = an EOA the trader controls. The Privy master signs Hyperliquid `approveAgent`, granting that EOA agent authority over the vault's Hyperliquid account.
3. The trader repeats step 2 with a **different** `apiWalletAddress` and a **different** `agentName` on each call. Because Hyperliquid only displaces a prior agent when the name matches, no earlier grant is revoked: the vault's Hyperliquid account now carries several concurrent trader-controlled agents. Each call may also pass a plausible `serviceName`, so the audit log attributes the grants to a legitimate consumer.
4. The owner notices and revokes trust: removes the trader (`setTrader` to a new address) and/or deactivates the vault. Both are DB-only.
5. **Every** planted EOA retains Hyperliquid agent authority and can keep placing orders on the vault's account until Hyperliquid's native agent expiry – the owner has no in-product way to revoke any of them, and after deactivation cannot even displace one by re-approving under its name.

Recommendation

1. **Add a revoke path and call it on lifecycle changes.** Implement Hyperliquid agent de-authorization on the wrapper and invoke it when a trader is removed or a vault is deactivated, withdrawing every outstanding grant for the vault rather than only the latest.
2. **Derive** `agentName` **server-side** from the authenticated consumer and/or the vault, so re-approval genuinely displaces the prior agent instead of installing an additional concurrent one.

Team Response

Fixed.

Non-Atomic Custody Workflows: Partial Failure Strands Funds and Orphans Vault State

Description

Two multi-step custody workflows execute as a sequence of independent awaits with no transaction boundary and no compensating action on partial failure, so a failure partway through leaves funds or state in an inconsistent, manually recoverable condition.

`withdrawToEvm` (`bridge`, `bridge.service.ts:79-147`). The withdrawal is three hops:

1. `spotSend` bridges USDC from HyperCore spot to the agent's HyperEVM address (`:101-107`);
2. poll HyperEVM for settlement (`:117`);
3. Privy-signed ERC-20 `transfer` from the agent to the caller's `evmDestination` (`:127-136`).

If hop 2 times out or hop 3 fails, hop 1 has already moved funds to the agent's EVM address. The code acknowledges this – on non-settlement it returns `BRIDGE_NOT_SETTLED` with *"The spotSend succeeded – funds may still arrive; retry the transfer leg separately"* (`:119-125`). The funds are not lost, but they are left mid-workflow and require a separate manual step to recover; there is no automatic completion or reversal.

`create` (`vaults`, `vaults.service.ts:45-146`). Vault creation performs, in order: provision a Privy agent wallet (`:54`), deploy the vault on-chain via the factory (`:59-78`), then insert the DB row (`:114-131`). These are not wrapped in a transaction and there is no cleanup on failure. If the on-chain deploy succeeds but the DB insert fails, the result is an on-chain vault plus a live Privy wallet with **no corresponding DB row** – an orphaned custody resource the service no longer tracks. Likewise a Privy wallet is provisioned even if the deploy later fails.

Scope note. This is a robustness/custody-integrity defect, not an attack: it does not assume a malicious owner/admin and requires no attacker – it is triggered by ordinary partial failures (RPC timeout, Hyperliquid latency, DB error).

Location of Affected Code

File: [src/bridge/bridge.service.ts#L79-L147](#)

File: [src/vaults/vaults.service.ts#L45-L146](#)

Impact

On partial failure of a custody workflow, funds or state are left inconsistent and require manual reconciliation: bridged funds stranded at the agent's EVM address awaiting a manual transfer-leg retry, or an on-chain vault / Privy wallet orphaned with no DB record. No funds are automatically lost and no attacker is required, but the divergence between on-chain/exchange state and the service's records can cause failed operations, double-processing on naive retries, and untracked custody resources.

Recommendation

1. **Persist workflow state before external side effects.** Record an intent row ahead of the Privy, chain, and exchange calls in `create` and `withdrawToEvm`, so a partial failure can be reconciled or resumed idempotently instead of leaving an orphaned wallet, an untracked vault, or funds stranded mid-bridge.

Team Response

Fixed.

M-02

MEDIUM

FIXED

swap-and-transfer **Moves the Total USDC Balance Rather Than Sale Proceeds**

Description

`doSwapSpotToUsdcAndTransferToPerp` sells a spot token for USDC and then moves the proceeds to perp. To determine "the proceeds," it reads the agent's **total** USDC balance *after* the sale and transfers that entire amount:

```
const after = await this.info.spotClearinghouseState({ user });
const usdc = after.balances.find((b) => b.coin === 'USDC');
const usdcReceived = usdc ? usdc.total : '0'; // <-- total balance, not delta
...
const transfer = await client.usdClassTransfer({ amount: usdcReceived, toPerp: true });
```

It never subtracts the **pre-sale** USDC balance (which is read earlier as `before`, but only used for a sufficiency check). Any USDC the agent already held in spot is therefore swept into perp along with the sale proceeds, and the returned `usdcReceived` overstates the sale by the pre-existing balance.

Concretely (demonstrated): agent holds 100 USDC in spot; selling the token yields 50 USDC; `after.total` = 150; the service transfers **150** to perp and reports `usdc_received: 150`, when this operation produced 50.

A second facet of the same USDC-accounting weakness:

- **Hardcoded 6 decimals** for USDC amount encoding (`signing.service.ts:122` , `bridge.service.ts:21`) - correct for USDC today, but an unchecked assumption baked into the transfer/accounting path.

Scope note. Owner-triggered (`swapAndTransfer` → `loadVaultAsOwner`); correctness/accounting defect, not an attack, and does not assume a malicious admin.

Location of Affected Code

File: [src/hyperliquid/hyperliquid.service.ts#L240-L247](#)

File: [`src/signing/signing.service.ts#L122`] (<https://github.com/pear-protocol/pear-vault-service/tree/2146d619466025a16a38f0ea387d14229b324614/src/signing/signing.service.ts#L122>)

File: [src/bridge/bridge.service.ts#L21](#)

Impact

The swap-and-transfer path moves more USDC to perp than the sale produced (sweeping any pre-existing spot USDC) and reports an inflated `usdc_received` to the consumer. Both are the vault's own funds, so there is no direct external theft; the harm is accounting integrity - a caller (e.g. `pear-hl-engine`) that trusts `usdc_received` for crediting, share/NAV computation, or reconciliation will over-count by the pre-existing balance, and an unintended sweep of spot USDC into perp can disrupt positions that relied on the spot balance.

Recommendation

1. **Transfer the delta, not the total.** Compute `usdcReceived = after.total - before.total` , transfer exactly that amount, and return it as `usdc_received` , so pre-existing spot USDC is neither swept nor double-counted.

Team Response

Fixed.

M-03

MEDIUM

FIXED

Events Indexer Has No Reorg Protection or Confirmation Depth

Description

`EventsIndexer.tick()` indexes `PearVault_Deposit` / `PearVault-Withdrawal` logs into `vault_events` , which is the table served by `GET /vaults/vaultAddress/transactions` and `GET /vault/transactions` as user-facing deposit/withdrawal history.

The indexer reads to the chain head returned by `getBlockNumber()` [:47] with **no confirmation depth**, then advances its checkpoint to that head [:61-65]. Because `startFrom` is unconditionally `checkpoint + 1` [:44-46], the scan is strictly forward-only: once a block range has been passed, it is never re-read under any circumstance. There is no rewind, no reorg detection, and no block-hash tracking that would allow one – the checkpoint table persists a block *number* and nothing else [0003_vault_events/migration.sql:28-33].

When HyperEVM reorgs a range the indexer has already passed, two things happen and neither self-corrects:

1. Events from the **orphaned** blocks remain in `vault_events` permanently, and continue to be served as user history.
2. Events from the **replacement** blocks are never indexed, because the checkpoint is already beyond them.

The `(tx_hash, log_index)` unique constraint does not mitigate this. A reorged-out transaction and its replacement have **different hashes**, so `skipDuplicates` sees two distinct rows rather than a conflict – it prevents double-insertion of the *same* log, not divergence between the DB and the canonical chain.

The same forward-only property means any log that becomes visible in an already-scanned range for a non-reorg reason (a lagging or load-balanced RPC node that had not yet served the log when the range was read) is also lost permanently.

Scope note. This is a data-integrity defect, not an attack: it requires no attacker and does not assume a malicious admin. It is triggered by ordinary chain behaviour.

Location of Affected Code

File: [src/events/events.indexer.ts#L44-L46](#)

File: [src/events/events.indexer.ts#L47](#)

File: [src/events/events.indexer.ts#L61-L65](#)

File: [src/events/events.indexer.ts#L133](#)

File: [prisma/schema.prisma#L89](#)

File: [prisma/migrations/0003_vault_events/migration.sql#L28-L33](#)

Impact

Vault deposit/withdrawal history diverges permanently from on-chain state after any reorg affecting an already-indexed range – reporting deposits that were orphaned and omitting deposits that actually settled. The divergence is silent (no error, no warning) and unrecoverable without manual intervention, since no code path re-reads a passed range.

Consumers reading these endpoints for user balance display, accounting, reconciliation, or support tooling will act on incorrect history. `vault_events` is a read/reporting surface rather than an authorization or fund-movement input – no funds move as a direct result – and the practical exposure scales with HyperEVM's real reorg depth and frequency, which is not established from this repository.

Proof of Concept

1. The chain head is block 100. Block 95 contains a `PearVault_Deposit` of 500 in tx `0xAAA...`. The indexer runs, records the deposit, and sets `last_block_number = 100`.
2. HyperEVM reorgs blocks 95-100. On the new canonical chain, the 500 deposit never happened; block 96 instead contains a different `PearVault_Deposit` of 900 in tx `0xBBB...`. The head returns to 100.
3. The indexer ticks again. `startFrom = 101 > 100`, so it returns immediately without re-reading the reorged range. It never will.
4. `GET /vaults/vaultAddress/transactions` now reports a 500 deposit that does not exist on chain, and omits the 900 deposit that does.

Recommendation

1. **Index behind finality.** Scan only to the RPC's `finalized` block tag where available (`[ getBlockNumber({ blockTag: 'finalized' }) ]`), otherwise to `getBlockNumber() - N` with `N` exposed as config alongside the existing `VAULT_EVENTS_*` settings, so the checkpoint never advances over blocks that can still be reorganised.
2. **Detect reorgs and rewind.** Persist the block hash alongside `last_block_number`, compare it on each tick, and on mismatch walk back to a matching ancestor, delete the `vault_events` rows above it, and reset the checkpoint. Without stored hashes, a reorg is undetectable in principle.

Team Response

Fixed.

M-04

MEDIUM

FIXED

NAV Subsystem Is Entirely Non-Functional: `multicall3` Is Not Configured on the HyperEVM Chain Object

Description

`ChainService` builds the HyperEVM chain object with viem's `defineChain`, correctly noting that viem has no HyperEVM built-in. However, `defineChain` constructs a chain object **from the literal it is given** – it does not merge viem's built-in contract registry. The literal declares `id`, `name`, `nativeCurrency` and `rpcUrls`, but no `contracts`, so `chain.contracts` is `undefined`.

`MulticallReader.readNav` then calls `publicClient.multicall()` without an explicit `multicallAddress`. Viem resolves the `multicall3` address from `chain.contracts.multicall3`, finds nothing, and throws:

```
ChainDoesNotSupportContract: Chain "HyperEVM" does not support contract "multicall3".
```

`readNav` does not catch it. The exception propagates to `NavPoller.tick()`, whose `try/catch` logs 'NAV poller tick failed' and swallows it. The poller then waits for the next interval and repeats.

Verification

Evaluated against the **installed** viem (`2.49.3` - `package.json` pins `^2.7.0`, which resolves forward) using the exact chain literal from `chain.service.ts:29-34`:

```
chain.contracts = undefined
multicall THREW: ChainDoesNotSupportContract | Chain "HyperEVM" does not support contract "multicall3".
```

This is deterministic and environment-independent – it follows from the chain object alone, not from RPC state (zero RPC calls are made; viem throws while resolving the `multicall3` address, before any `eth_call`).

Location of Affected Code

File: [src/chain/chain.service.ts#L29-L34](#)

File: [src/chain/multicall.ts#L42](#)

File: [src/nav/nav.poller.ts#L48](#)

File: [src/nav/nav.poller.ts#L68-L70](#)

Impact

No NAV snapshot is ever written (once at least one *active* vault exists – with no active vaults `NavPoller.tick` returns before `multicall`, `nav.poller.ts:45`). `vault_nav_snapshots` remains empty, and therefore:

- `GET /vaults/vault_address/nav/latest` → **HTTP 404 NAV_UNAVAILABLE** (the controller converts the missing row into a 404, `nav.controller.ts:12` – it does not return `null`)
- `GET /vaults/vault_address/nav` returns `[]`
- `GET /vaults/vault_address/nav/stats` returns `samples: 0, return_pct: null`
- The TimescaleDB continuous aggregates (`vault_nav_5m`, `_1h`, `_1d`) have no source rows

The failure is silent apart from one log line per tick. There is no NAV spec in the test suite, which is the likely reason it shipped.

Note this also makes moot any concern about NAV snapshot block-number accuracy or poller re-entrancy – that code never completes.

Recommendation

1. Declare the contract on the chain object:

```
contracts: { multicall3: { address: '0xcA11bde05977b3631167028862bE2a173976CA11' } }
```

Confirmed deployed on HyperEVM: `eth_getCode` at that address returns bytecode on chain id 999.

2. Do not swallow poller exceptions unconditionally – surface configuration errors loudly instead of letting the subsystem fail silently.

Team Response

Fixed.

M-05

MEDIUM

FIXED

Events Indexer Has No Per-Vault Backfill: Migrated Vaults Miss Their Pre-Insertion History

Description

The indexer tracks progress with **one global checkpoint row** (`vault_event_checkpoint`, `id:1`) that only ever moves forward. Each tick reads every vault address, then scans `checkpoint+1 → head`. Nothing establishes a per-vault starting point, and nothing backfills when a vault is added.

Two consequences follow.

1. Migrated vaults miss their pre-insertion history (primary)

`POST /sync/migrate-vault` is the documented cutover path from `pear-hl-engine` (`docs/api.md:86`). It upserts vaults that already have deposit/withdrawal history on chain, at blocks **below** the current checkpoint. Because the checkpoint only advances, those blocks are never scanned again. (Events emitted *after* the vault is inserted – and after the checkpoint – are indexed normally; the loss is specifically the on-chain history that predates the insertion. "Never receive history" is too broad.)

Precondition: migration runs after the indexer has advanced its checkpoint past those historical blocks. The indexer auto-starts on boot (`onModuleInit`), while migration is a manual cutover step.

Newly-created vaults are unaffected, since they have no prior events. That asymmetry likely masked the defect during testing.

2. The default start block is genesis (secondary)

`VAULT_EVENTS_START_BLOCK` defaults to `0` in the zod schema, so **omitting the variable entirely** yields a genesis scan on first boot. Every other risky value in that schema is either required (no default) or safely defaulted; this one has an actively harmful default.

With `VAULT_EVENTS_BLOCK_BATCH=1000` and HyperEVM at ~41,761,584 blocks (queried live), a first pass issues roughly **41,762 sequential `getLogs` calls inside a single tick**, each filtered across every vault address. Against a shared public RPC, this takes hours or dies on rate limits.

This half is self-recovering – the checkpoint advances per batch (`61-65`), so a crash resumes rather than restarting. It is the migrated-vault data loss that is the substantive impact.

Interaction worth noting: the masking works the other way around. `getLogs` is filtered to the vault addresses present in the DB *when the tick begins* (`events.indexer.ts:39,54-55`), so a scan cannot capture a vault that has not been migrated yet. It is *migration completing before the scan reaches those blocks* that incidentally captures the history – if the migrated address is in the DB before the checkpoint passes its historical range, blocks in `[checkpoint, history_end]` are picked up (blocks already scanned before insertion are still lost). A genesis scan completing *before* migration captures nothing for that vault. Either way it is timing luck, not design.

Location of Affected Code

File: [src/config/app.config.ts#L45](#)

File: [src/events/events.indexer.ts#L43-L48](#)

File: [src/events/events.indexer.ts#L52-L67](#)

File: [src/vaults/vaults.controller.ts#L103-L120](#)

Impact

Migrated vaults are permanently missing their **pre-insertion** deposit/withdrawal history on `GET /vaults/vault_address/transactions` – for exactly the vaults the cutover exists to onboard. Users see no history for funds they deposited before migration. No authorization or balance depends on this data; the impact turns on how important transaction history is as a customer and accounting surface.

Recommendation

1. **Track progress per vault.** Key the checkpoint on `vault_address`, or enqueue a one-shot backfill when a vault row is inserted, starting from that vault's deployment block.
2. **Remove the `0` default for `VAULT_EVENTS_START_BLOCK`** and require it explicitly, set to the vault factory's deployment block.

Team Response

Fixed.

M-06

MEDIUM

FIXED

Raw Prisma BigInt Values Make Non-Empty Transaction-History and nav/latest Responses Unserializable

Description

Prisma maps a PostgreSQL `BIGINT` column to a native JavaScript `bigint`. Two models carry such a column, and both are returned to HTTP callers without conversion.

```
model VaultEvent {  
  // ...  
  blockNumber BigInt @map("block_number")  
}
```

`EventsService.list()` returns the Prisma records unmodified:

```
return this.prisma.vaultEvent.findMany({  
  where,  
  orderBy: [{ blockNumber: 'desc' }, { logIndex: 'desc' }],  
  take: Math.min(filter.limit ?? 100, 1000),  
  skip: filter.offset ?? 0,  
});
```

and the controller passes that straight to Nest:

```
@Get('vaults/vaultAddress/transactions')  
byVault(@Param('vaultAddress') vaultAddress: string, @Query() q: ListEventsDto) {  
  return this.events.list({ ... });  
}
```

No response DTO, mapping function, serializer interceptor, or JSON replacer converts `blockNumber` to a string. `bootstrap()` applies only `helmet()` and a `ValidationPipe`. A repository-wide search for `BigInt.prototype.toJSON`, `JSON.stringify`, `Interceptor`, and `replacer` across `src/` returns no matches, so nothing anywhere in the application supplies the missing serialization.

`JSON.stringify` has no representation for a native `bigint` and throws:

```
TypeError: Do not know how to serialize a BigInt
```

The consequences follow directly:

- An **empty** result set serializes normally as `[]`.
- Any result containing **at least one** `VaultEvent` reaches the response layer holding a native `bigint` and fails serialization, producing HTTP 500.
- `GET /vaults/vaultAddress/nav/latest` has the same defect via `VaultNavSnapshot.blockNumber`.

The NAV **range** endpoint is unaffected: its raw SQL casts the column explicitly with `block_number:text` (`nav.service.ts:76-88`), which is the correct handling and is already present one function away from the defective path.

Location of Affected Code

File: [prisma/schema.prisma#L64](#)

File: [prisma/schema.prisma#L83](#)

File: [src/events/events.service.ts#L19-L24](#)

File: [src/events/events.controller.ts#L10-L17](#)

File: [src/nav/nav.service.ts#L62-L66](#)

File: [src/nav/nav.controller.ts#L9-L13](#)

File: [src/main.ts#L7-L18](#)

Impact

Deposit and withdrawal history becomes unavailable as soon as the indexer holds data. The failure mode is inverted relative to normal testing: an empty database returns `200 []`, so the endpoint appears healthy in a fresh environment and breaks only once real data exists.

The same failure will surface on `GET /vaults/vaultAddress/nav/latest` once NAV snapshots begin to be written.

Impact is confined to API availability and incorrect status codes. Indexed data, authorization decisions, balances, and funds are unaffected, and an unauthenticated caller cannot cause persistent state corruption through this path.

Proof of Concept

1. The events indexer writes at least one `DEPOSIT` or `WITHDRAWAL` row for a vault.
2. Any caller requests `GET /vaults/Ox.../transactions`. The route requires no authentication.
3. Prisma returns records whose `blockNumber` is a native `bigint`.
4. Nest hands the object to the Express JSON response path.

5. Serialization throws `TypeError`, and the caller receives HTTP 500 instead of the history.
6. The failure repeats for every request that returns a non-empty result, for every vault and every consumer.

Recommendation

1. **Convert `BigInt` fields to decimal strings in an explicit response mapping**, in both `EventsService.list()` and `NavService.latest()`:

```
return rows.map([row] => [{ ...row, blockNumber: row.blockNumber.toString() }]);
```

Prefer explicit response mappers over a process-wide `BigInt.prototype.toJSON` modification, so the public representation of each field stays deliberate and typed.

Team Response

Fixed.

L-01

• LOW

FIXED

Internal API Key Compared in Non-Constant Time (Timing Side-Channel)

Description

The internal API key guard compares the caller-supplied key to the configured secret with a plain string inequality:

```
if (typeof provided !== 'string' || provided !== this.config.internalApiKey) {  
  throw new UnauthorizedException('Invalid internal API key');  
}
```

JavaScript's `!==` on strings short-circuits at the first differing byte, so verification time correlates with the length of the matching prefix. This is a classic timing side-channel: an attacker who can measure response latency across many `/sync/*` requests could, in principle, recover `INTERNAL_API_KEY` byte-by-byte without ever guessing it whole. Remote timing attacks over the network are noisy and require a large, low-jitter sample, and the guard protects internal cutover routes – but the comparison should still be constant-time, since the key it protects authorizes vault-state migration (`/sync/migrate-vault`).

This does not assume a malicious admin; the attacker is any party able to reach the `/sync/*` endpoints and measure timing.

Location of Affected Code

File: [src/auth/api-key-guard.ts#L21](#)

Recommendation

Compare the keys in constant time. Use `crypto.timingSafeEqual` over equal-length buffers (guard against length differences first), e.g. hash both sides to a fixed length or length-check before the timing-safe compare. Do not use `===` / `!==` on the raw secret.

Team Response

Fixed.

L-02

● LOW

FIXED

Unvalidated `.vaultAddress` **Path Param**
Reaches `getAddress()` **and Returns HTTP 500**

Description

`GET /vaults/vaultAddress/transactions` takes `vaultAddress` from the URL path and forwards it to `EventsService.list`, which calls viem's `getAddress(filter.vaultAddress)` to build the query. The **query** params on this route are validated (`ListEventsDto` marks `vaultAddress` / `userAddress` as `@IsEthereumAddress`), but the **path** param is not – there is no validation pipe or DTO on `@Param('vaultAddress')`. viem's `getAddress` throws `InvalidAddressError` on any non-address string, and nothing catches it, so the request returns **HTTP 500** instead of a `400 Bad Request`.

Example: `GET /vaults/not-an-address/transactions` → unhandled `getAddress` throw → 500. The route is unauthenticated (see L-03, Unauthenticated Reads of Transaction History and NAV), so any caller can trigger it. Impact is low – a single erroring request, not resource exhaustion – but 500s on attacker-controlled input are incorrect error handling and pollute monitoring and alerting.

This does not assume a malicious admin; the trigger is a malformed path segment from any caller.

Location of Affected Code

File: [src/events/events.controller.ts#L10-L13](#)

File: [src/events/events.service.ts#L17-L18](#)

Recommendation

Validate the path param as an Ethereum address before use – apply a validation pipe / DTO to `@Param('vaultAddress')` (as other controllers do), or normalize-and-validate inside the service and throw a `BadRequestException` on failure, so malformed input yields `400`, not `500`. Apply the same treatment to any other route that passes a raw path param into `getAddress`.

Team Response

Fixed.

L-03

• LOW

ACKNOWLEDGED

Unauthenticated Reads of Transaction History and NAV; NAV Range Unbounded

Description

Auth in this service is opt-in per controller (`@UseGuards(ConsumerAuthGuard)`), and there is no global guard. `EventsController` and `NavController` carry no guard, so the following are served to fully anonymous callers:

- `GET /vaults/:vaultAddress/transactions` and `GET /vault/transactions` – deposit/withdrawal history, **filterable by an arbitrary** `userAddress` query param, so any caller can read any user's vault-activity history.
- `GET /vaults/:vaultAddress/nav` (`latest` / `range` / `stats`) – NAV series and statistics.

Two facets:

- **Unauthenticated information disclosure.** Vault transaction history and NAV are readable without any credentials, and the transaction endpoint lets the caller target a specific `userAddress`.
- **Unbounded read on NAV range.** `NavService.range` issues a raw SQL query with no row cap (`nav.service.ts:76-88`); the result set is bounded only by the caller's `from` / `to`. A wide date window returns the entire NAV history in one response. (The transaction endpoint is capped at 1000 rows/page but is fully enumerable via `offset`.)

The exposed data is vault activity and NAV rather than keys or credentials (the `privyId` disclosure is covered by C-01, Cross-Vault Privy Signer Substitution). It shares a root cause with the missing-global-guard posture. This does not assume a malicious admin; the caller is any anonymous party.

Location of Affected Code

File: [src/events/events.controller.ts#L4-L27](#)

File: [src/nav/nav.controller.ts#L5-L23](#)

File: [src/nav/nav.service.ts#L69-L88](#)

Recommendation

1. **Authenticate these reads.** Apply `ConsumerAuthGuard`, and restrict `userAddress` filtering to the authenticated caller's own address so history cannot be enumerated for arbitrary users.
2. **Bound the NAV range.** Cap the number of returned points server-side so a single request cannot pull unbounded history.

Team Response

Acknowledged.

L-04

LOW

FIXED

`initialUsdcDeposit` **Is Accepted and Validated on Vault Creation, Then Silently Ignored**

Description

`CreateVaultDto` declares an optional `initialUsdcDeposit` field and validates it as a positive number. `VaultsService.create()` - which provisions the Privy wallet, deploys the vault via the factory, and inserts the DB row - **never reads it**. It is the only DTO field in the codebase that is validated and then dropped. No deposit is made, no transfer is initiated, and nothing is persisted to record that one was requested.

The interaction with the global validation pipe is what makes this more than dead code. `main.ts:10-12` configures `forbidNonWhitelisted: true`, so an unrecognised property in a request body is rejected with `400 Bad Request`. Because `initialUsdcDeposit` **is** declared on the DTO, it is whitelisted and accepted silently. A caller therefore cannot distinguish "this parameter is supported" from "this parameter is ignored" by observation - the request succeeds, the vault is created, and the response (`vaults.service.ts:137-145`) contains no field that would reveal the deposit did not happen.

This is reinforced by the service's own documentation: `docs/api.md:154` shows `initial_usdc_deposit: 100` in the canonical "pear-pro-backend creates a new vault" call flow, presenting it as a working parameter of the create request.

Scope note. This is a correctness / API-contract defect. It does not assume a malicious admin and requires no attacker; the trigger is an ordinary, documented create request.

Location of Affected Code

File: [src/vaults/dto/create-vault.dto.ts#L33-L36](#)

File: [src/vaults/vaults.service.ts#L45-L146](#)

File: [src/main.ts#L10-L12](#)

File: [docs/api.md#L154](#)

Recommendation

1. **Decide the contract and make the code match it.** Either implement the initial deposit in `create()`, or remove `initialUsdcDeposit` from `CreateVaultDto` so supplying it is rejected with `400` by the existing `forbidNonWhitelisted` pipe – and drop `initial_usdc_deposit` from `docs/api.md:154` accordingly.

Team Response

Fixed.

L-05

LOW

FIXED

Poll Intervals Above 59 Seconds Are Silently Ignored and Misreported in Logs

Description

Both pollers interpolate a configured interval into the **seconds field** of a six-field cron expression:

```
const cronExpr = `*/${seconds} * * * *`;
```

The seconds field has a range of 0-59. A step larger than the range matches only `0`, so any value above 59 silently degrades to once per minute, and values that do not divide 60 produce an uneven cadence.

Verification

Run against the pinned `cron@3.2.1`:

INTERVAL SECONDS	EXPRESSION	ACTUAL FIRINGS
15	<code>* / 15 * * * *</code>	<code>:15 :30 :45 :00</code> – correct
45	<code>* / 45 * * * *</code>	<code>:45 :00 :45 :00</code> – 45s then 15s
60	<code>* / 60 * * * *</code>	every 60s – correct
90	<code>* / 90 * * * *</code>	every 60s

INTERVAL SECONDS	EXPRESSION	ACTUAL FIRINGS
120	<code>* /120 *****</code>	every 60s
300	<code>* /300 *****</code>	every 60s

No exception is raised at any value.

Location of Affected Code

File: [src/nav/nav.poller.ts#L31-L36](#)

File: [src/events/events.indexer.ts#L28-L32](#)

File: [src/config/app.config.ts#L34](#)

File: [src/config/app.config.ts#L44](#)

Impact

The configured interval is silently ignored above 59 seconds, and the startup log actively misreports it – `NavPoller` logs `NAV poller started – every 300s` while polling every 60s. An operator raising the interval to reduce RPC load or avoid provider rate limits receives up to 5× the intended request volume and no signal that the setting did not take effect.

Downstream, NAV read failures are skipped silently (`multicall.ts:51-53`), so the consequence surfaces as unexplained gaps in NAV history rather than as a configuration error.

The shipped defaults (60 and 30) both happen to be divisors of 60 and behave correctly, so this only manifests once an operator changes the value – which is precisely when it is least expected.

Recommendation

1. Use `setInterval` with a millisecond period, or emit a minute-field expression when `seconds >= 60`, and bound both values in the zod schema so any value that cannot be expressed exactly is rejected.

Team Response

Fixed.

Vault Service Applies No Independent JWT Age Cap or Revocation

Description

The consumer auth guard verifies a consumer's RS256 JWT but enforces a maximum token age **only when** the consumer's registry entry defines `maxTokenAgeSeconds` (`consumer-auth.guard.ts:62`). That field is optional in the schema (`consumers.config.ts:16`) and is **absent for the engine consumer** `pear-protocol-api` in `config/consumers.json`. The guard also does not require an `exp` claim and has no token-revocation / `jti` mechanism.

The engine (`pear-protocol-api`) issues standard RS256 user-session JWTs with a **15-minute** expiry by default (env-configurable), and there is **no revocation/deny-list on the vault-service side** - verification is stateless, and the only kill-switch is removing the consumer's key from the registry. In normal operation the replay window is therefore bounded to ~15 minutes by the engine's `exp`, which `jwt.verify` enforces. What remains is a defense-in-depth gap.

Consequences for engine-issued tokens:

- The vault service applies **no independent age cap** for this consumer - it trusts the engine's `exp` entirely. If that `exp` were ever raised (it is env-configurable) or a token were issued without `exp`, the guard would accept it for that full (or unbounded) lifetime, because `maxTokenAgeSeconds` is absent for `pear-protocol-api`.
- A leaked token **cannot be invalidated** server-side within its ~15-minute window; there is no `jti` / deny-list, so a stolen token remains usable against fund-touching routes until it expires (short of the nuclear registry-key removal).

In scope, not admin-malicious. The precondition is **token theft/leakage** (an ordinary external precondition). A stolen token lets the holder act as the token's `address` for up to the ~15-minute window.

Location of Affected Code

File: [src/auth/consumer-auth.guard.ts#L62](#)

File: [src/config/consumers.config.ts#L16](#)

File: [config/consumers.json](#)

Impact

A leaked engine JWT is replayable for its ~15-minute lifetime with no server-side revocation, so a stolen token grants up to ~15 minutes of impersonation on the victim's vault operations, unrevocable except by removing the consumer key. Bounded and requiring token theft, but the vault service's total reliance on the engine's `exp` (no independent cap, no `exp`-required check, no revocation) means the exposure scales directly with any future change to the engine's token lifetime – it is not defended in depth on this side.

Recommendation

1. **Make** `maxTokenAgeSeconds` **required** in the zod consumer schema, so every consumer, including the engine, is bounded, and set a short value for `pear-protocol-api`.
2. **Require an** `exp` **claim** and cap the accepted lifetime independently of the token's self-declared value.

Team Response

Fixed.

L-07

LOW

FIXED

No Code-Side Rate Limiting on Operator-Funded Vault Creation

Description

The client has confirmed that ingress restriction is *"today... a design assumption, not enforced from the code side ... we'll verify the ingress rules on the deployed environment and lock it down there."* Since ingress is not enforced in code today, the code-side gap is reported here; the operator has committed to enforcing it at the infrastructure layer.

`POST /vaults` has no per-caller or per-owner rate limit, and every accepted call provisions a Privy wallet and initiates an **operator-funded** on-chain vault deploy (paid by `VAULT_DEPLOYER_PRIVATE_KEY`). `dto.leader` must equal the caller, which constrains *whose* vault is created, not *how many*.

The only thing bounding who can reach this route is the network boundary – which the client confirms is **currently a design assumption, not enforced in code** (the repo ships only a Docker container; VPC/security-group/ingress rules live in infra config and are not yet applied). So the service should not rely solely on that boundary; it currently has no code-side control on this operator-cost operation.

Location of Affected Code

File: [src/vaults/vaults.service.ts#L45-L78](#)

File: [src/vaults/vaults.controller.ts#L29-L34](#)

File: [src/main.ts](#)

File: [package.json](#)

Impact

Low (defense-in-depth). If the network boundary is not enforced (its current state) and a caller can reach `POST /vaults`, they can loop it – each iteration spending deployer gas and Privy quota with no cap – draining the deployer account and (once it is empty) halting new vault creation. This is a resource-consumption / operator-cost exposure, not a fund-theft or data-integrity issue, and the operator has committed to enforcing ingress at the infra layer, which would remove the external reachability. It is raised as defense-in-depth: the code path should be safe even if the network boundary is misconfigured or bypassed.

Recommendation

1. **Add per-caller and per-owner rate limiting on** `POST /vaults` in the application itself, rather than relying on network ingress alone.

Team Response

Fixed.

L-08

● LOW

FIXED

Upstream Provider Error Text Is Returned Verbatim in API Responses

Description

Error handlers across the signing, Hyperliquid, and vault-creation paths catch a third-party failure and pass the upstream exception message directly into the HTTP response body. No global exception filter is registered in `main.ts` – `bootstrap()` applies `helmet()` and a `ValidationPipe` only – so nothing redacts these values on the way out.

The messages originate from the Privy SDK, the Hyperliquid client, and the HyperEVM RPC provider. Error strings from these systems routinely carry request identifiers, internal endpoint paths, wallet identifiers, provider-side rejection reasons, and node implementation detail.

`privy.client.ts:78` is the sharpest instance – a `signTypedData` failure returns the Privy SDK's message under an explicitly named `privy_error` key:

```
} catch (err) {
  this.logger.error('Privy signTypedData failed for ${privyId}', err as Error);
  throw new InternalServerErrorException({
    error: 'SIGNING_FAILED',
    privy_error: (err as Error).message,
  });
}
```

This route is reachable by any authenticated vault owner or trader. The same shape repeats six times in `hyperliquid.service.ts` and once in `vaults.service.ts:83`, where a failed factory deploy returns the RPC provider's error text.

Location of Affected Code

File: [src/signing/privy.client.ts#L57](#)

File: [src/signing/privy.client.ts#L78](#)

File: [src/signing/privy.client.ts#L126](#)

File: [src/hyperliquid/hyperliquid.service.ts#L61](#)

File: [src/hyperliquid/hyperliquid.service.ts#L102](#)

File: [src/hyperliquid/hyperliquid.service.ts#L141](#)

File: [src/hyperliquid/hyperliquid.service.ts#L162](#)

File: [src/hyperliquid/hyperliquid.service.ts#L179](#)

File: [src/hyperliquid/hyperliquid.service.ts#L202](#)

File: [src/vaults/vaults.service.ts#L83](#)

File: [src/main.ts#L7-L18](#)

Impact

Information disclosure to authenticated API callers about internal infrastructure, third-party provider identities, and upstream failure modes. There is no direct compromise and no privilege gain. The practical effect is reconnaissance value: a caller probing the signing and bridging paths learns provider-side detail that a generic error would not reveal, which shortens the work of mapping the integration surface before attempting a more substantive attack.

Proof of Concept

1. An authenticated caller (owner or trader on any vault) issues `POST /vaults/:vaultAddress/sign` with a payload the Privy API will reject.
2. `PrivyClientWrapper.signTypedData` catches the SDK error and returns HTTP 500 with a body of the form `{ "error": "SIGNING_FAILED", "privy_error": "<upstream SDK message>" }`.
3. The caller reads provider-side detail from the response – the failure reason, and whatever identifiers or endpoint information the SDK included in its message – without any access to the service's logs or infrastructure.
4. Repeating the pattern across `/sign`, `/sign/transfer`, the bridge routes, and `POST /vaults` yields a picture of the upstream providers, their rejection semantics, and the service's integration surface.

Recommendation

1. **Register a global exception filter** that maps internal errors to a stable application code and a generic message, and remove the `privy_error` and `reason: (err as Error).message` passthroughs at `privy.client.ts:57,78,126`, `hyperliquid.service.ts:61,102,141,162,179,202`, and `vaults.service.ts:83`. The upstream text is already captured by the existing `logger.error` calls.

Team Response

Fixed.

I-01

● INFO

FIXED

`SignTransferDto.amount` **Bypasses Decimal Validation, Silently Signing Zero-Value Transfers**

Description

The codebase defines a shared decimal guard and applies it consistently to amount fields:

```
const DECIMAL = /^^\d+(\.\d+)?$/;
```

`SignTransferDto.amount` is the exception – it carries only `@IsString() @IsNotEmpty()`. The value flows unchecked into `buildTransferTypedData`, which calls `parseUnits(input.amount, 6)` and encodes the result as an ERC-20 `transfer` argument.

Verification

Run against the **installed** viem (`2.49.3` – `package.json` pins `^2.7.0`, which resolves forward), mirroring the call in `signing.service.ts:119-123`:

INPUT		PARSEUNITS(SIX, 6)	ENCODEFUNCTIONDATA
"10"	10000000		OK
"0.00000001"	0		OK
"0.99999999"	1000000 (rounds)		OK
"-5"	-5000000 (no throw here)		throws IntegerOutOfRangeException
"abc"	throws InvalidDecimalNumberError		-
"1e9"	throws InvalidDecimalNumberError		-

(Error classes are version-dependent: `viem 2.49.3` throws `InvalidDecimalNumberError` for `"abc"` and `"1e9"`, and `"-5"` does **not** throw at `parseUnits` - the throw moves to `encodeFunctionData`.)

Location of Affected Code

File: [src/signing/dto/sign.dto.ts#L53-L55](#)

File: [src/signing/signing.service.ts#L122](#)

File: [src/bridge/dto/bridge.dto.ts](#)

File: [src/agent/dto/agent.dto.ts](#)

Impact

A correctness and API-hygiene defect, not a security issue as it stands.

Silent truncation to zero (primary). An amount with more precision than the assumed 6 decimals - `"0.00000001"` - parses to `0n` and encodes successfully, so the service returns a validly signed EIP-712 authorization for **zero value** with HTTP 200 and no warning. But `/sign/transfer` only *returns a signature* - it does not execute a transfer; the caller supplies the amount themselves; and a zero-value authorization loses no funds and grants no excess authority. There is no downstream executor shown that treats the returned signature as a completed/settled transfer while relying on the human-readable amount. Absent such a consumer, the harm is a useless signature returned to the caller who requested it. It does compound the hardcoded-6-decimals assumption in M-02 (`swap-and-transfer` Moves the Total USDC Balance Rather Than Sale Proceeds); for an 18-decimal token, ordinary amounts truncate the same way.

Unhandled exceptions (secondary). Malformed or negative amounts throw inside the service and surface as HTTP 500 rather than 400. Same class as L-02 (Unvalidated `.vaultAddress` Path Param), different parameter.

Recommendation

1. **Require** `parsedAmount > 0` **after parsing**. The shared `DECIMAL` regex does not prevent this - `/^\d+(\.\d+)?$/` matches `"0.00000001"` - so validate precision and reject values that truncate to zero at the token's decimals.
2. **Return** `400` **rather than** `500` for malformed or negative input.

Team Response

Fixed.

I-02

● INFO

FIXED

`/health` **Reports** `privy_connected` **Without Ever Contacting Privy**

Description

`GET /health` returns `{ status, db_connected, privy_connected, uptime_seconds }` and is the service's liveness/dependency signal [`src/health/health.controller.ts#L21-L37`].

The two dependency checks are not equivalent in strength:

- `db_connected` is a genuine check - it executes `SELECT 1` against Postgres and reports the result [`health.controller.ts#L24-L29`].
- `privy_connected` is not. It returns `PrivyClientWrapper.isReady()` [`health.controller.ts#L30`], which returns a private `ready` boolean [`src/signing/privy.client.ts#L39-L41`]. That flag is set to `true` unconditionally in `onModuleInit` immediately after the `PrivyClient` constructor runs [`privy.client.ts#L33-L37`], and is never set to `false` again anywhere in the class.

`new PrivyClient(appId, appSecret)` only constructs a local SDK object; it performs no network round-trip and validates no credentials. So `privy_connected: true` establishes only that the process reached `onModuleInit` - it is true for the entire lifetime of the process regardless of Privy's actual availability.

Consequences:

- A Privy outage, revoked/rotated app secret, DNS failure, or network partition to Privy is reported as `status: 'ok'`. Every signing route will fail with `500 SIGNING_FAILED` while health stays green.
- Because `status` is `dbOk && privyOk`, the Privy term can never contribute a `degraded` result. The endpoint's aggregate status is effectively a DB check alone.
- Orchestrators and uptime monitors consuming `/health` will not drain, restart, or alert on a service whose highest-value dependency - the one this service exists to own - is unreachable.

The class doc comment at `privy.client.ts#L23` describes a *"boot-time connectivity check (used by /health)"*. No such check is performed; the comment overstates what the code does, which is likely how the gap survived.

This is Informational: no attacker, no fund exposure, and no incorrect authorization – the impact is degraded observability at the moment it matters most.

Location of Affected Code

File: [src/health/health.controller.ts#L21-L37](#)

File: [src/health/health.controller.ts#L24-L29](#)

File: [src/health/health.controller.ts#L30](#)

File: [src/signing/privy.client.ts#L23](#)

File: [src/signing/privy.client.ts#L33-L37](#)

File: [src/signing/privy.client.ts#L39-L41](#)

Recommendation

1. **Make the check real.** Perform a lightweight Privy API call and report its outcome, cached with a short TTL, and allow the flag to return to `false` when calls fail.
2. **Or rename the field** to `privy_initialized` if the intent is only that the client was constructed, so operators are not misled about which dependency was actually verified.

Team Response

Fixed.



Your security partner, for the long term

[SHIELDIFY.ORG](https://shieldify.org) · [X.COM/SHIELDIFYSEC](https://x.com/shieldifysec) · [LINKEDIN](#)